

pyAmpliCol 0.1.0

Methodology and Performance Report

Rikkert Frederix

Valentin Hirschi

Timea Vitos

Release 0.1.0

Abstract

This report defines the public theory, validation, and performance methodology for **pyAmpliCol**, release 0.1.0. It fixes the canonical process coverage for the built-in Standard Model, an externally loaded Standard Model, a scalar contact model, and a scalar-gravity model. Its tables are a reproducible publication surface for measurements produced from the developer installation and recorded with their effective configuration, environment, artifacts, logs, and independent-reference provenance.

Contents

1	Scope	2
2	Current Recursion and Model Lowering	2
3	Colour and Runtime Semantics	3
4	Benchmark Methodology	3
5	Standard-Model Process Matrices	5
6	Z-plus-Jets Evaluator Ladders	19
7	Colour-Singlet Model Ladders	22
8	Interpretation	22

1 Scope

pyAmpliCol generates tree-level scattering-amplitude artifacts and evaluates them through the Rusticol runtime. The Python layer resolves models and processes, constructs a shared-current directed acyclic graph (DAG), lowers bounded stages to Symbolica evaluators, and writes a schema-v3 process artifact. The runtime loads that artifact and provides summed and resolved evaluation through one physics contract to Python, Rust 2021, C++17, and Fortran 2008 clients.

The report has three purposes:

1. specify the observables and conditions required for a publishable benchmark;
2. preserve a stable matrix of representative process families and colour approximations; and
3. separate measured data from prose by rendering all result tables from validated JSON caches.

An `N/A` entry means that the current cache has no recorded measurement for that slot. It is neither a zero, a failure, nor a value carried over from another implementation. Explicit failure labels such as `T/O`, `RAM>800G`, `VALIDATION FAILED`, and `ERROR` indicate measured campaign outcomes with saved logs.

2 Current Recursion and Model Lowering

2.1 Shared-current DAG

For an external subset I , particle state t , and local spin or flow label χ , an off-shell current has the schematic recursion

$$J_{t,\chi}^\alpha(I) = P^\alpha_\beta(t, p_I) \sum_{I=A \cup B} V^\beta_{\gamma\delta}(t_A, t_B \rightarrow t) J_{t_A, \chi_A}^\gamma(A) J_{t_B, \chi_B}^\delta(B), \quad p_I = \sum_{i \in I} p_i. \quad (1)$$

Current identity includes the particle, external labels, helicity ancestry, chirality or spin state, flavour and charge flow, colour state, momentum mask, and any auxiliary-current kind. Contributions with the same complete identity share one node. A topological sweep therefore evaluates each retained current once per phase-space point and reuses it in all dependent amplitudes.

External UFO or serialized JSON sources are compiled to the same canonical model representation as the process generator consumes. Particle metadata, interactions, coupling orders, Lorentz tensors, colour tensors, propagators, and runtime parameters come from that representation. UFO directories contain trusted executable Python; serialized JSON is the portable data-only source.

Optimizations on the external-model path are proof-gated rather than selected by particle names or PDG values. Exact normalized colour and Lorentz expressions, propagator metadata, and coupling relations certify current reflection, helicity, and reusable-kernel identities. For overlapping Standard-Model validation processes, the resulting current, interaction, amplitude-root, and colour-sector counts must match the built-in compatibility model after applying the same independently inferred perturbative-order envelope. An external compiler may retain fewer evaluator calls only when an additional exact kernel-equivalence certificate proves that reuse.

2.2 Contact terms and spin two

A vertex with valence above three is lowered to a deterministic tree of non-propagating auxiliary currents. The construction inserts the original coupling once and introduces no pole. Direct contraction of the original contact tensor is the validation oracle for the scalar-contact ladder.

The scalar-gravity ladder exercises rank-two external states and momentum-dependent interactions. For a massless graviton, the helicity-two polarization tensor is represented as

$$\epsilon_{\pm 2}^{\mu\nu} = \epsilon_{\pm}^\mu \epsilon_{\pm}^\nu. \quad (2)$$

Validation checks symmetry, transversality, tracelessness, normalization, and agreement between ordinary and higher-precision evaluation.

3 Colour and Runtime Semantics

The process matrices use three colour contracts:

LC	Leading-colour ordered amplitudes with physical colour-flow selectors.
NLC	The leading term plus the first retained $1/N_c^2$ -suppressed contribution.
Full	The complete supported sparse colour metric for the generated amplitude basis.

The optimized `Runtime.evaluate()` path returns one summed matrix element per point.

The resolved method `Runtime.evaluate_resolved()` exposes LC values with axes (`point`, `helicity`, `flow`). NLC and full-colour artifacts expose one already contracted colour component per helicity. Summing all non-point axes must reproduce the optimized total. Public selectors use physical helicity and flow identifiers; graph-local colour-sector identifiers are not report inputs.

All language bindings load the same process manifest and evaluator payloads. Cross-language agreement therefore validates the runtime interfaces and data layout. A separately generated reference is required for an independent physics comparison.

The default JIT artifact embeds a self-contained SymJIT application for f64 evaluation. Rusticol executes that payload through the separate MIT-licensed SymJIT runtime without importing Symbolica or consulting its runtime license state. Higher-precision Python evaluation and artifacts generated with the ASM or C++ evaluator backends retain an explicit Symbolica runtime capability.

4 Benchmark Methodology

4.1 Configuration contract

Each measurement begins from an immutable resolved configuration. Unless a table variant says otherwise, the report contract uses the release defaults: requested SymJIT optimization level 3, evaluator batch size 128, output chunk size 128, stage-local parameter layout, ten Horner iterations, backend-default common-pair elimination, a ten-second benchmark target, two warmup runs, and at least five timed samples. The process matrices always record requested level 3 in their machine-readable benchmark contract; only explicitly labelled comparison rows, such as JIT level 1 in the dedicated Z ladder, override it. SymJIT may apply its documented backend fallback independently of the requested level. Requested and effective configurations are stored separately so resource or license adjustments remain visible.

Generation time is wall time for one explicit `Generator.generate()` operation starting from the declared model source and ending with a committed, loadable artifact. Model-source conversion, process generation, evaluator compilation, and API-bundle emission must be identified separately when they are measured separately. Existing artifacts may be benchmarked, but their generation slot remains N/A unless a matching generation measurement is available.

Runtime measurements use `BenchmarkRunner` and the summed `Runtime.evaluate()` observable. A cache entry records sample count, wall and evaluator time per point, standard deviation, standard error, relative standard error, effective settings, and environment. Comparisons are valid only for the same process, colour accuracy, parameters, phase-space ensemble, precision, selectors, and normalization.

4.2 Numerical validation

The matrix element and its independent reference are evaluated at the same deterministic points with the same parameter card. Shared-core language checks use relative tolerance 10^{-12} and absolute tolerance 10^{-15} . Independent reference comparisons use relative tolerance 10^{-8} , with higher precision used to diagnose unstable points. A populated result must also satisfy the resolved-sum identity and preserve expected structural zeros.

No timing is accepted as a numerical validation result. Conversely, a valid matrix element does not populate generation or runtime fields without a matching timing run.

Publication workflow. The report service validates all cache entries, renders every table in a staging directory, optionally compiles the PDF there, and publishes the JSON, TeX, and PDF set only after all steps succeed. Failed refreshes leave the previous published set in place. Physics generation and benchmarking are not implicit report operations.

5 Standard-Model Process Matrices

The canonical matrix spans electroweak vector production, charged-current and leptonic channels, heavy-quark production, pure gluons, multi-boson final states, Higgs-associated final states, and three- and four-quark-line cases. Here n is the number of final-state particles. LC coverage retains the established grid through $n = 9$, with the four bounded families ending at $n = 8$. NLC and full-colour grids retain $n = 1, \dots, 5$; cells below a family’s minimum physical multiplicity remain explicitly marked N/A and carry an applicability flag in JSON.

The external-SM matrices use the same process definitions, colour contracts, and numerical input parameters as the built-in matrices. Their model source is the packaged Standard Model loaded through the external JSON/UFO compilation path, so the comparison isolates model-source and generic-lowering behaviour without changing the requested physics or the original Fortran AmpliCol reference.

5.1 Built-in SM LC process matrix

ID base process	n=1	n=2	n=3
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
gen O3 one-flow, hel. sum	N/A	N/A	N/A
	N/A	N/A	N/A
gen O3 all-flows, one-hel	N/A	N/A	N/A
	N/A	N/A	N/A
run O3 one-flow, hel. sum	N/A	N/A	N/A
	N/A	N/A	N/A
run O3 all-flows, one-hel	N/A	N/A	N/A
	N/A	N/A	N/A

Built-in SM LC process matrix (continued)

ID base process	n=4	n=5	n=6
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
gen O3 one-flow, hel. sum	N/A N/A	N/A N/A	N/A N/A
gen O3 all-flows, one-hel	N/A N/A	N/A N/A	N/A N/A
run O3 one-flow, hel. sum	N/A N/A	N/A N/A	N/A N/A
run O3 all-flows, one-hel	N/A N/A	N/A N/A	N/A N/A

Built-in SM LC process matrix (continued)

ID base process	n=7	n=8	n=9
1 $d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2 $u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3 $d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4 $u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5 $d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6 $gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7 $d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8 $gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9 $d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10 $d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11 $d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12 $d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13 $d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14 $d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
gen O3 one-flow, hel. sum	N/A	N/A	N/A
	N/A	N/A	N/A
gen O3 all-flows, one-hel	N/A	N/A	N/A
	N/A	N/A	N/A
run O3 one-flow, hel. sum	N/A	N/A	N/A
	N/A	N/A	N/A
run O3 all-flows, one-hel	N/A	N/A	N/A
	N/A	N/A	N/A

Matrix cells use the old matrix layout: the upper line shows original Fortran AmpliCol generation time and the pyAmpliCol JIT O3 generation ratio; the lower line shows original Fortran AmpliCol runtime and the pyAmpliCol JIT O3 wall|evaluator runtime ratios. Ratios are pyAmpliCol divided by AmpliCol, so values below one mean pyAmpliCol is faster.

5.2 Built-in SM NLC process matrix

ID	base process	n=1	n=2	n=3
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: gen		N/A	N/A	N/A
summary: run wall		N/A	N/A	N/A
summary: run eval		N/A	N/A	N/A

Built-in SM NLC process matrix (continued)

ID	base process	n=4	n=5
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: gen		N/A	N/A
summary: run wall		N/A	N/A
summary: run eval		N/A	N/A

Matrix cells use the old matrix layout: the upper line shows original Fortran AmpliCol generation time and the pyAmpliCol JIT O3 generation ratio; the lower line shows original Fortran AmpliCol runtime and the pyAmpliCol JIT O3 wall|evaluator runtime ratios. Ratios are pyAmpliCol divided by AmpliCol, so values below one mean pyAmpliCol is faster.

5.3 Built-in SM full-colour process matrix

ID	base process	n=1	n=2	n=3
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: gen		N/A	N/A	N/A
summary: run wall		N/A	N/A	N/A
summary: run eval		N/A	N/A	N/A

Built-in SM full-colour process matrix (continued)

ID	base process	n=4	n=5
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: gen		N/A	N/A
summary: run wall		N/A	N/A
summary: run eval		N/A	N/A

Matrix cells use the old matrix layout: the upper line shows original Fortran AmpliCol generation time and the pyAmpliCol JIT O3 generation ratio; the lower line shows original Fortran AmpliCol runtime and the pyAmpliCol JIT O3 wall|evaluator runtime ratios. Ratios are pyAmpliCol divided by AmpliCol, so values below one mean pyAmpliCol is faster.

5.4 UFO-SM LC process matrix

ID	base process	n=1	n=2	n=3
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
gen O3 one-flow, hel. sum		N/A	N/A	N/A
gen O3 all-flows, one-hel		N/A	N/A	N/A
run O3 one-flow, hel. sum		N/A	N/A	N/A
run O3 all-flows, one-hel		N/A	N/A	N/A

UFO-SM LC process matrix (continued)

ID	base process	n=4	n=5	n=6
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
gen O3 one-flow, hel. sum		N/A	N/A	N/A
gen O3 all-flows, one-hel		N/A	N/A	N/A
run O3 one-flow, hel. sum		N/A	N/A	N/A
run O3 all-flows, one-hel		N/A	N/A	N/A

UFO-SM LC process matrix (continued)

ID	base process	n=7	n=8	n=9
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6	$g\bar{g} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8	$g\bar{g} \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
gen O3 one-flow, hel. sum		N/A	N/A	N/A
gen O3 all-flows, one-hel		N/A	N/A	N/A
run O3 one-flow, hel. sum		N/A	N/A	N/A
run O3 all-flows, one-hel		N/A	N/A	N/A

Matrix cells use the old matrix layout: the upper line shows original Fortran AmpliCol generation time and the pyAmpliCol JIT O3 generation ratio; the lower line shows original Fortran AmpliCol runtime and the pyAmpliCol JIT O3 wall|evaluator runtime ratios. Ratios are pyAmpliCol divided by AmpliCol, so values below one mean pyAmpliCol is faster.

5.5 UFO-SM NLC process matrix

ID	base process	n=1	n=2	n=3
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: gen		N/A	N/A	N/A
summary: run wall		N/A	N/A	N/A
summary: run eval		N/A	N/A	N/A

UFO-SM NLC process matrix (continued)

ID	base process	n=4	n=5
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: gen		N/A	N/A
summary: run wall		N/A	N/A
summary: run eval		N/A	N/A

Matrix cells use the old matrix layout: the upper line shows original Fortran AmpliCol generation time and the pyAmpliCol JIT O3 generation ratio; the lower line shows original Fortran AmpliCol runtime and the pyAmpliCol JIT O3 wall|evaluator runtime ratios. Ratios are pyAmpliCol divided by AmpliCol, so values below one mean pyAmpliCol is faster.

5.6 UFO-SM full-colour process matrix

ID	base process	n=1	n=2	n=3
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A
summary: gen		N/A	N/A	N/A
summary: run wall		N/A	N/A	N/A
summary: run eval		N/A	N/A	N/A

UFO-SM full-colour process matrix (continued)

ID	base process	n=4	n=5
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A
summary: gen		N/A	N/A
summary: run wall		N/A	N/A
summary: run eval		N/A	N/A

Matrix cells use the old matrix layout: the upper line shows original Fortran AmpliCol generation time and the pyAmpliCol JIT O3 generation ratio; the lower line shows original Fortran AmpliCol runtime and the pyAmpliCol JIT O3 wall|evaluator runtime ratios. Ratios are pyAmpliCol divided by AmpliCol, so values below one mean pyAmpliCol is faster.

6 Z-plus-Jets Evaluator Ladders

The dedicated $d\bar{d} \rightarrow Z + (n-1)g$ ladders compare the independent reference with JIT level 1, JIT level 3, ASM O3, and C++ O3 evaluator configurations. They retain final-state multiplicities $n = 1, \dots, 9$ for the built-in model and $n = 1, \dots, 6$ for the external-model campaign. Variant overrides are stored as typed dotted configuration paths in the cache rather than inferred from table labels.

Worked $d\bar{d} \rightarrow Zg$ example

For $n = 2$, the explicit process is

$$d(p_1) \bar{d}(p_2) \rightarrow Z(p_3) g(p_4).$$

The report labels source particles in process order. The old leading-colour reference order for the selected-flow timing is therefore (2, 4, 1, 3): the colour line enters through $\bar{d}$, traverses the gluon, and exits through d , while the colour-singlet Z remains in the public ordering only to keep source labels unambiguous. Internally, the colour-sector match is the coloured word (2, 4, 1).

pyAmpliCol builds one shared-current DAG for the selected process. Source nodes carry particle, momentum, helicity, flavour, and colour-flow labels; composite nodes are keyed by the full current identity and the external-label mask. When two diagrams need the same current, the DAG contains one node and downstream amplitude roots reuse its value. In the selected-flow/helicity-sum workload the generator keeps the colour sector compatible with (2, 4, 1) and sums source helicities. In the all-flows/fixed-helicity workload it keeps all LC colour flows but fixes one DAG-supported source-helicity assignment. The generated artifact serializes the stages and evaluator payloads; Rusticol loads that artifact, executes `Runtime.evaluate()` inside the native core, and `BenchmarkRunner` records both the native repeated wall time and the sum of evaluator-call timings.

6.1 Dedicated $d\bar{d} \rightarrow Z +$ Gluon Performance

n process	route	setup	selected flow, helicity sum			all flows, fixed helicity		
			gen [s]	wall [us/pt]	eval [us/pt]	gen [s]	wall [us/pt]	eval [us/pt]
1 $d^- \bar{d}^+ \rightarrow Z + (1-1)*g$	reference	AmpliCol	N/A	N/A	N/A	N/A	N/A	N/A
1 $d^- \bar{d}^+ \rightarrow Z + (1-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
1 $d^- \bar{d}^+ \rightarrow Z + (1-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
1 $d^- \bar{d}^+ \rightarrow Z + (1-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
1 $d^- \bar{d}^+ \rightarrow Z + (1-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
2 $d^- \bar{d}^+ \rightarrow Z + (2-1)*g$	reference	AmpliCol	N/A	N/A	N/A	N/A	N/A	N/A
2 $d^- \bar{d}^+ \rightarrow Z + (2-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
2 $d^- \bar{d}^+ \rightarrow Z + (2-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
2 $d^- \bar{d}^+ \rightarrow Z + (2-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
2 $d^- \bar{d}^+ \rightarrow Z + (2-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
3 $d^- \bar{d}^+ \rightarrow Z + (3-1)*g$	reference	AmpliCol	N/A	N/A	N/A	N/A	N/A	N/A
3 $d^- \bar{d}^+ \rightarrow Z + (3-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
3 $d^- \bar{d}^+ \rightarrow Z + (3-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
3 $d^- \bar{d}^+ \rightarrow Z + (3-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
3 $d^- \bar{d}^+ \rightarrow Z + (3-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
4 $d^- \bar{d}^+ \rightarrow Z + (4-1)*g$	reference	AmpliCol	N/A	N/A	N/A	N/A	N/A	N/A
4 $d^- \bar{d}^+ \rightarrow Z + (4-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
4 $d^- \bar{d}^+ \rightarrow Z + (4-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
4 $d^- \bar{d}^+ \rightarrow Z + (4-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
4 $d^- \bar{d}^+ \rightarrow Z + (4-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
5 $d^- \bar{d}^+ \rightarrow Z + (5-1)*g$	reference	AmpliCol	N/A	N/A	N/A	N/A	N/A	N/A
5 $d^- \bar{d}^+ \rightarrow Z + (5-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
5 $d^- \bar{d}^+ \rightarrow Z + (5-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
5 $d^- \bar{d}^+ \rightarrow Z + (5-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
5 $d^- \bar{d}^+ \rightarrow Z + (5-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A

n process	route	setup	selected flow, helicity sum			all flows, fixed helicity		
			gen [s]	wall [us/pt]	eval [us/pt]	gen [s]	wall [us/pt]	eval [us/pt]
6 d d ⁻ > Z + (6-1)*g	reference	AmpliCol	N/A	N/A	N/I	N/A	N/A	N/A
6 d d ⁻ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
6 d d ⁻ > Z + (6-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
6 d d ⁻ > Z + (6-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
6 d d ⁻ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/I	N/A	N/A	N/A
7 d d ⁻ > Z + (7-1)*g	reference	AmpliCol	N/A	N/A	N/I	N/A	N/A	N/A
7 d d ⁻ > Z + (7-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
7 d d ⁻ > Z + (7-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
7 d d ⁻ > Z + (7-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
7 d d ⁻ > Z + (7-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/I	N/A	N/A	N/A
8 d d ⁻ > Z + (8-1)*g	reference	AmpliCol	N/A	N/A	N/I	N/A	N/A	N/A
8 d d ⁻ > Z + (8-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
8 d d ⁻ > Z + (8-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
8 d d ⁻ > Z + (8-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
8 d d ⁻ > Z + (8-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/I	N/A	N/A	N/A
9 d d ⁻ > Z + (9-1)*g	reference	AmpliCol	N/A	N/A	N/I	N/A	N/A	N/A
9 d d ⁻ > Z + (9-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
9 d d ⁻ > Z + (9-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
9 d d ⁻ > Z + (9-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
9 d d ⁻ > Z + (9-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/I	N/A	N/A	N/A

6.2 UFO-SM Dedicated $d\bar{d} \rightarrow Z + \text{Gluon}$ Performance

n process	route	setup	selected flow, helicity sum				all flows, fixed helicity					
			gen [s]	vs blt-in	wall [us/pt]	vs blt-in	eval [us/pt]	gen [s]	vs blt-in	wall [us/pt]	vs blt-in	eval [us/pt]
1 $d\bar{d} \rightarrow Z + (1-1)*g$	reference	AmpliCol	N/A		N/A		N/I	N/A		N/A		N/A
1 $d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
1 $d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
1 $d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
1 $d\bar{d} \rightarrow Z + (1-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/I	N/A	N/A	N/A	N/A	N/A
2 $d\bar{d} \rightarrow Z + (2-1)*g$	reference	AmpliCol	N/A		N/A		N/I	N/A		N/A		N/A
2 $d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
2 $d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
2 $d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
2 $d\bar{d} \rightarrow Z + (2-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/I	N/A	N/A	N/A	N/A	N/A
3 $d\bar{d} \rightarrow Z + (3-1)*g$	reference	AmpliCol	N/A		N/A		N/I	N/A		N/A		N/A
3 $d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
3 $d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
3 $d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
3 $d\bar{d} \rightarrow Z + (3-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/I	N/A	N/A	N/A	N/A	N/A
4 $d\bar{d} \rightarrow Z + (4-1)*g$	reference	AmpliCol	N/A		N/A		N/I	N/A		N/A		N/A
4 $d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
4 $d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
4 $d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
4 $d\bar{d} \rightarrow Z + (4-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/I	N/A	N/A	N/A	N/A	N/A
5 $d\bar{d} \rightarrow Z + (5-1)*g$	reference	AmpliCol	N/A		N/A		N/I	N/A		N/A		N/A
5 $d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
5 $d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
5 $d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
5 $d\bar{d} \rightarrow Z + (5-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/I	N/A	N/A	N/A	N/A	N/A
6 $d\bar{d} \rightarrow Z + (6-1)*g$	reference	AmpliCol	N/A		N/A		N/I	N/A		N/A		N/A
6 $d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
6 $d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
6 $d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
6 $d\bar{d} \rightarrow Z + (6-1)*g$	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/I	N/A	N/A	N/A	N/A	N/A

Z tables use the old performance-summary layout: reference rows are original Fortran AmpliCol, pyAmpliCol rows show generation, wall, and evaluator timings with ratios against the reference where available. Variant overrides are machine-readable in JSON.

7 Colour-Singlet Model Ladders

The scalar-contact ladder retains $n = 2, \dots, 8$ final scalars. It tests generic high-valence contact decomposition, identical-particle normalization, and source parity without Standard-Model-specific kernels. The scalar-gravity ladder retains $n = 2, \dots, 4$ final gravitons and exercises spin-two wavefunctions, tensor propagators, and momentum-dependent vertices. Since all external particles in these fixtures are colour singlets, LC, NLC, and full colour reduce to the same unit colour contraction.

For these ladders, the relative-difference row is $|M_{f64} - M_{hp}| / \max(|M_{hp}|, 10^{-300})$, where M_{hp} is the same point re-evaluated through the higher-precision diagnostic path. It is a numerical stability check, not a performance ratio.

7.1 Scalar-contact ladder

metric	scalar_0 scalar_0 > n*scalar_0						
	n = 2	n = 3	n = 4	n = 5	n = 6	n = 7	n = 8
generation [s]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
wall [μ s/pt]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
evaluator [μ s/pt]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
matrix element	N/A	N/A	N/A	N/A	N/A	N/A	N/A
relative diff. vs hp	N/A	N/A	N/A	N/A	N/A	N/A	N/A

7.2 Scalar-gravity ladder

metric	scalar_0 scalar_0 > n*graviton		
	$n = 2$	$n = 3$	$n = 4$
generation [s]	N/A	N/A	N/A
wall [μ s/pt]	N/A	N/A	N/A
evaluator [μ s/pt]	N/A	N/A	N/A
matrix element	N/A	N/A	N/A
relative diff. vs hp	N/A	N/A	N/A

8 Interpretation

This report establishes coverage, methodology, and the current measured cache state. Performance regression gates should be enabled only after the same developer-installed artifact has produced a complete, reviewed baseline on named hardware. Consumers should treat every N/A entry as unavailable and should not interpolate, copy, or estimate a value from neighbouring multiplicities or model profiles.

The machine-readable caches under `docs/results/` are authoritative for table completeness and measurement provenance. The TeX inputs and this PDF are derived publication artifacts.